

VisionInspect AI: Manufacturing Defect Detection & Quality Inspection System

Milestone 2 Documentation

Darapuneni Bhavya

TABLE OF CONTENTS

S.No	Title	Page No.
1	Introduction	1
2	Objectives of Milestone 2	1
3	Dataset Description	2
4	Dataset Preparation	3
5	Image Preprocessing	4-5
6	AI Model Training and Evaluation	6-9
7	AI Model Integration	10
8	Backend and Database Integration	11-12
9	Frontend Implementation	13-17
10	Conclusion and Future Enhancements	18

1. Introduction

In modern manufacturing industries, maintaining product quality is essential for ensuring customer satisfaction and reducing production losses. Traditional quality inspection methods rely on manual inspection, which is time-consuming and may produce inconsistent results due to human error. To overcome these limitations, industries are increasingly adopting Artificial Intelligence (AI) and Computer Vision technologies for automated quality inspection.

The **VisionInspect AI: Manufacturing Defect Detection & Quality Inspection System** is designed to automate the process of identifying defective and non-defective manufacturing products using image classification. In this project, the **MVTec AD** dataset is used to train a deep learning model capable of classifying products as **Defective** or **No Defect**. The trained model is integrated with a **FastAPI** backend and a **React.js** frontend, enabling users to upload manufacturing images and receive real-time prediction results along with confidence scores.

The integration of Artificial Intelligence into manufacturing not only improves inspection accuracy but also reduces inspection time and minimizes human intervention. Automated defect detection helps industries maintain consistent product quality, improve production efficiency, and reduce operational costs.

Milestone 2 focuses on developing and integrating the Artificial Intelligence pipeline of the project. It includes dataset preparation, image preprocessing, model training using **MobileNetV2**, model evaluation, and integration of the trained model with the application. This milestone forms the core of the system by enabling accurate and automated manufacturing defect detection.

2. Objectives of Milestone 2

The primary objective of Milestone 2 is to develop the Artificial Intelligence module of the **Manufacturing Defect Detection & Quality Inspection System**. The specific objectives are:

- To implement image preprocessing pipelines for preparing manufacturing images before model training.
- To generate image quality analysis reports based on the processed images.

- To build image analytics workflows for efficient inspection and defect analysis.
- To train a deep learning-based defect detection model using the **MobileNetV2** transfer learning architecture.
- To generate accurate defect predictions by classifying manufacturing images as **Defective** or **No Defect**.
- To integrate the trained AI model with the application for real-time defect prediction.
- To build an inspection monitoring workflow that supports automated quality inspection and production monitoring.
- To improve the efficiency, accuracy, and reliability of the manufacturing quality inspection process using Artificial Intelligence and Computer Vision.

3. Dataset Description

The **MVTec AD (Anomaly Detection)** dataset is used in the VisionInspect AI project for training and evaluating the defect detection model. It is one of the most widely used benchmark datasets for industrial anomaly detection and manufacturing quality inspection.

The dataset contains high-quality images of manufacturing products categorized into **Defective** and **No Defect** classes. These images represent various industrial products with different types of surface defects and normal product conditions. The dataset is used to train the deep learning model to accurately distinguish between defective and non-defective products.

The dataset provides diverse industrial images that help the model learn various defect patterns and improve its ability to identify manufacturing defects accurately. Its high-quality images and well-organized categories make it suitable for developing reliable AI-based quality inspection systems.

For model development, the dataset is prepared and organized into **Training, Validation, and Testing** subsets, which are discussed in the following section.

This well-organized dataset plays a significant role in developing an efficient and reliable AI-based defect detection system for manufacturing quality inspection.

4. Dataset Preparation

Dataset preparation is one of the most important steps in building a reliable deep learning model. In the VisionInspect AI project, the `prepare_dataset.py` script is developed to organize the **MVTec AD** dataset into a structured format suitable for model training and evaluation. The script automatically reads the images from the original dataset and prepares them for the training process.

The dataset is divided into three subsets: **Training**, **Validation**, and **Testing**. The training dataset is used to train the model, the validation dataset is used to monitor the model's performance during training and reduce overfitting, and the testing dataset is used to evaluate the final performance of the trained model on previously unseen images.

The script also creates the required folder structure and places the images into their respective categories, ensuring that the dataset is properly organized for efficient model training.

4.1 Dataset Preparation Workflow

1. Load the original MVTEC AD dataset.
2. Read images from the **Defective** and **No Defect** categories.
3. Randomly shuffle the dataset.
4. Split the images into **Training**, **Validation**, and **Testing** subsets.
5. Create the required folder structure.
6. Save the images into their respective folders.

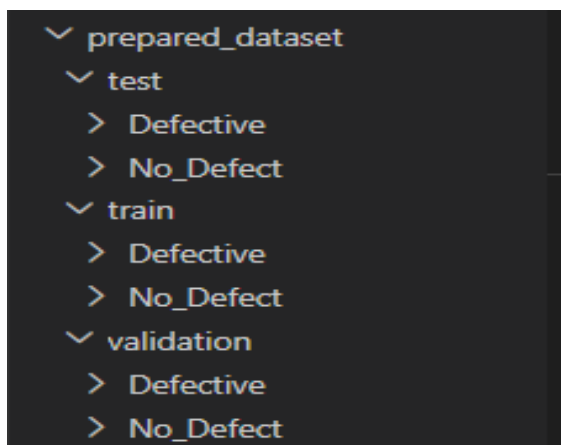


Figure 4.1: Prepared Dataset Folder Structure

The prepared dataset is organized into three subsets: **Training**, **Validation**, and **Testing**. Each subset contains two categories: **Defective** and **No_Defect**. This structured organization enables the AI model to learn from the training data, validate its performance during training, and evaluate its accuracy using unseen test images.

5. Image Preprocessing

Image preprocessing is an important step in the VisionInspect AI system, as it enhances the quality of the input image before it is analyzed by the AI model. Proper preprocessing helps improve the consistency of the input data and enables the model to identify defects more accurately.

In this project, the **preprocessing.py** module is implemented using the **OpenCV** library. The uploaded manufacturing image is first read and its original dimensions are obtained. The image is then resized to **256 × 256 pixels** to ensure a uniform input size. After resizing, the image is converted into **grayscale**, which simplifies image processing by reducing color information. Finally, **Gaussian Blur** is applied to remove noise and improve image quality before the image is passed to the AI model.

The processed image is saved in the **processed** folder, and important image details such as height, width, channels, and processed image path are returned to the backend. This preprocessing pipeline ensures that all input images are processed consistently before defect prediction.

5.1 Image Preprocessing Workflow

1. Read the uploaded manufacturing image.
2. Obtain the original image dimensions.
3. Resize the image to **256 × 256 pixels**.
4. Convert the resized image into grayscale.
5. Apply Gaussian Blur to reduce image noise.
6. Save the processed image in the **processed** folder.
7. Return the processed image information to the backend.

```
def preprocess_image(image_path, processed_folder):

    # Read image
    image = cv2.imread(image_path)

    if image is None:
        return None

    # Original dimensions
    height, width, channels = image.shape

    # Resize image
    resized = cv2.resize(image, (256, 256))

    # Convert to Grayscale
    gray = cv2.cvtColor(resized, cv2.COLOR_BGR2GRAY)

    # Remove noise using Gaussian Blur
    blur = cv2.GaussianBlur(gray, (5, 5), 0)

    # Save processed image
    filename = os.path.basename(image_path)
    processed_path = os.path.join(processed_folder, filename)
```

Figure 5.1: Implementation of preprocessing.py

The preprocessing.py module is implemented using the OpenCV library. It reads the uploaded image, resizes it to **256 × 256 pixels**, converts it into grayscale, applies Gaussian Blur for noise reduction, and saves the processed image. The processed image information is then returned to the backend for further AI model prediction.

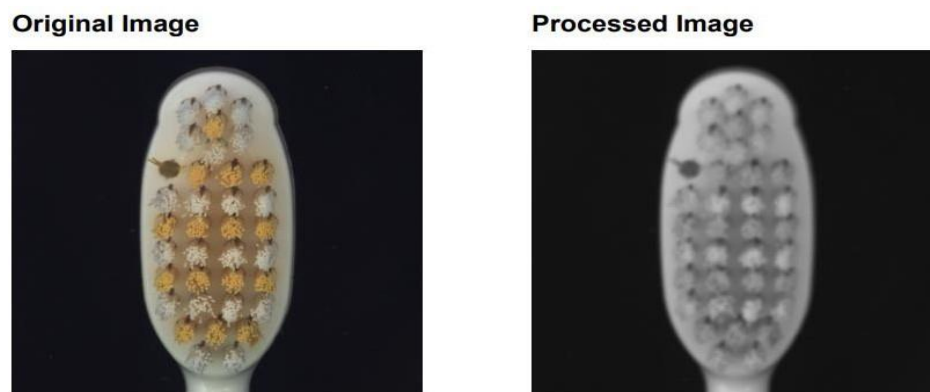


Figure 5.2: Original Image and Processed Image

The preprocessing pipeline converts the uploaded manufacturing image into a processed format suitable for AI model prediction. The processed image is resized, converted to grayscale, and filtered using Gaussian Blur, resulting in a cleaner image for accurate defect detection.

6. AI Model Training and Evaluation

6.1 AI Model Training

After preparing the dataset, the AI model was trained using the MobileNetV2 transfer learning architecture. The model was trained to classify manufacturing images into two categories: **Defective** and **No Defect**. During training, data augmentation techniques were applied to improve the model's performance. Early Stopping and Model Checkpoint were used to prevent overfitting and automatically save the best-performing model. The training process generated accuracy and loss graphs to monitor the learning performance of the model.

```
=====
Test Accuracy : 84.61%
Test Loss      : 0.3813
=====

=====
Training Completed Successfully!
Best Model Saved : best_model.keras
Accuracy Graph   : accuracy.png
Loss Graph       : loss.png
=====
```

Figure 6.1: Model Training Output

6.2 Training vs Validation Accuracy Graph

The **Training vs Validation Accuracy** graph shows how the model accuracy improved during each training epoch. Both the training accuracy and validation accuracy increased gradually, indicating that the model learned the image features effectively and achieved better classification performance.

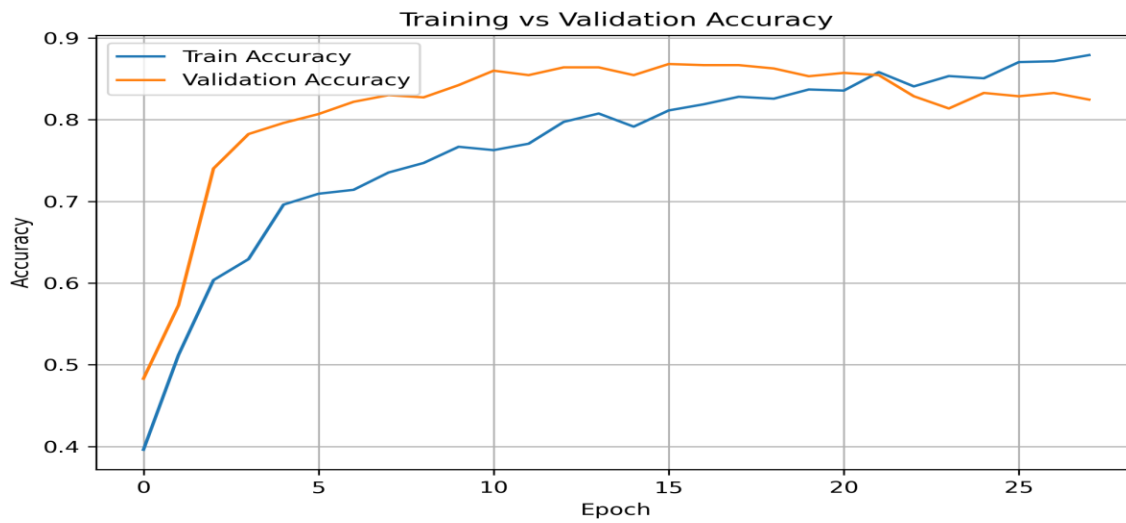


Figure 6.2: Training vs Validation Accuracy

6.3 Training vs Validation Loss Graph

The **Training vs Validation Loss** graph shows how the prediction error decreased during the training process. Both the training loss and validation loss reduced gradually over multiple epochs, indicating that the model improved its learning and classification performance.

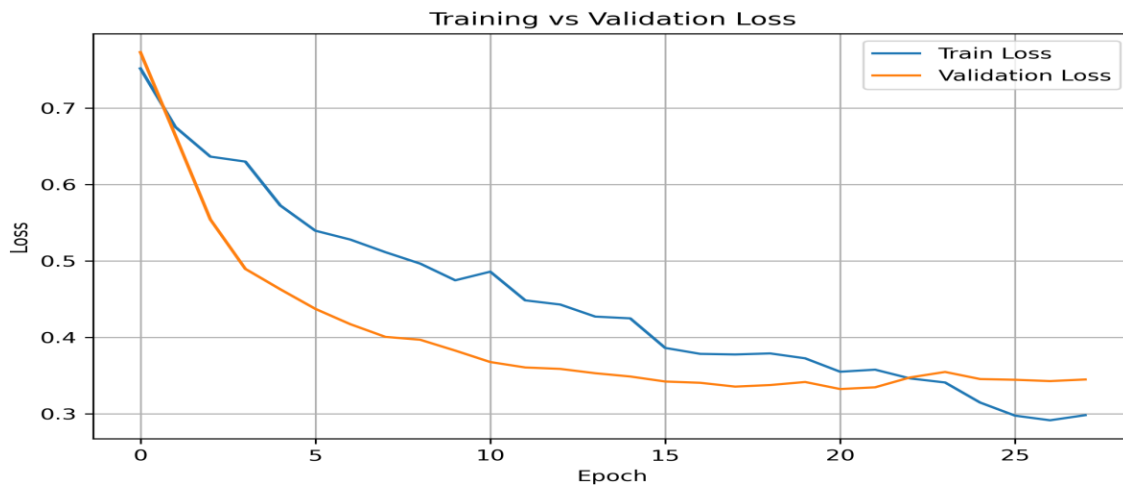


Figure 6.3: Training vs Validation Loss

6.4 Confusion Matrix

The confusion matrix provides a visual representation of the model's classification performance on the test dataset. It compares the actual class labels with the predicted class labels and shows the number of correctly and incorrectly classified images for both **Defective** and **No Defect** categories. This helps in analyzing the strengths and weaknesses of the trained classification model.

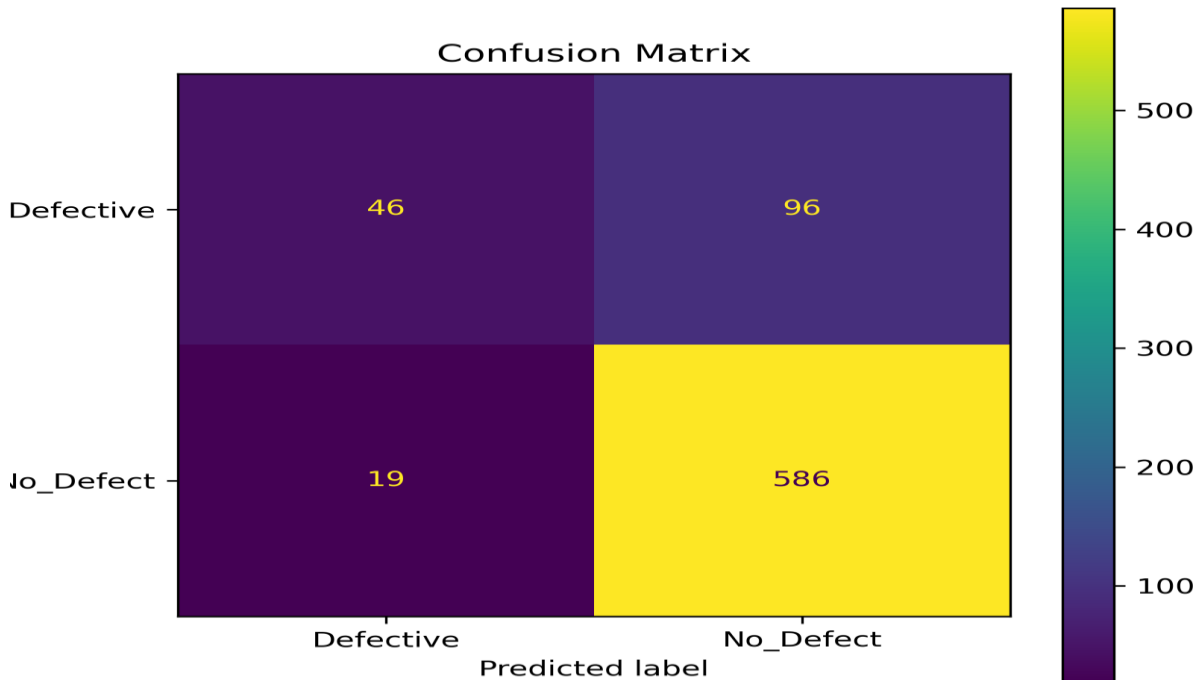


Figure 6.4: Confusion Matrix

6.5 Model Evaluation

After training, the model was evaluated using the test dataset. The evaluation generated the test accuracy, test loss, and classification report, which measure the overall performance of the trained model on unseen manufacturing images.

```

PS C:\Users\Mohan\Desktop\VisionInspectAI\training> python evaluate.py
=====
Test Loss      : 0.3813
Test Accuracy  : 84.61%
=====
24/24 [=====] - 48s 2s/step

Classification Report

              precision    recall  f1-score   support

 Defective      0.71      0.32      0.44       142
  No_Defect     0.86      0.97      0.91       605

 accuracy              0.85       747
 macro avg           0.78      0.65      0.68       747
 weighted avg        0.83      0.85      0.82       747

Confusion Matrix Saved : confusion_matrix.png
Classification Report Saved : classification_report.txt

Evaluation Completed Successfully!

```

Figure 6.5: Model Evaluation Output

6.6 Model Performance

Performance Metric	Value
Test Accuracy	84.61%
Test Loss	0.3813
Classification Type	Binary Classification
Classes	Defective, No Defect
Saved Model	best_model.keras

6.7 Summary

The AI model was successfully trained and evaluated using the MVTec AD dataset. The accuracy and loss graphs demonstrate the learning behavior of the model during training, while the confusion matrix and evaluation results confirm its ability to classify images as **Defective** or **No Defect**. The trained model was saved as **best_model.keras** and integrated with the FastAPI backend for real-time manufacturing defect classification.

7. AI Model Integration

The trained model is integrated with the backend application to perform real-time manufacturing defect detection. After the training process is completed, the best-performing model is saved as **best_model.keras** and loaded into the backend using TensorFlow.

The AI model receives the preprocessed manufacturing image as input and performs image classification to identify whether the product is **Defective** or **No Defect**. Before prediction, the input image is resized to **224 × 224 pixels**, converted into a numerical array, normalized, and prepared in a format suitable for the MobileNetV2 model.

After prediction, the system calculates the confidence score and returns both the predicted class and confidence value to the backend. These results are then sent to the frontend, where they are displayed to the user along with the processed image. This integration enables the application to provide fast, reliable, and real-time AI-based manufacturing defect detection.

7.1 AI Model Integration Workflow

1. Load the trained **best_model.keras** model.
2. Receive the preprocessed image from the backend.
3. Resize the image to **224 × 224 pixels**.
4. Normalize the image and prepare it for prediction.
5. Perform defect prediction using the trained AI model.
6. Classify the image as **Defective** or **No Defect**.
7. Calculate the confidence score.
8. Return the prediction result to the backend.
9. Display the prediction and confidence score in the frontend application.

8. Backend and Database Integration

The backend of the **VisionInspect AI** system is developed using the **FastAPI** framework. It acts as the communication layer between the **React.js frontend**, the trained AI model, and the **MongoDB** database. The backend receives uploaded manufacturing images, performs image preprocessing, invokes the trained AI model for prediction, stores the inspection details in the database, and returns the prediction results to the frontend.

The system uses **MongoDB** to store user information and inspection history. Two collections, namely **users** and **inspection_history**, are created to manage user authentication data and inspection records. After every successful prediction, important details such as the filename, image dimensions, prediction result, confidence score, processed image size, and inspection date are automatically stored in the database for future reference.

The backend also provides a **Dashboard API**, which retrieves important inspection statistics from MongoDB. It calculates the total number of inspections, defective products, non-defective products, overall inspection accuracy, and recent inspection records. These statistics are displayed on the dashboard to provide users with a clear summary of the inspection process. This integration enables the system to perform real-time manufacturing defect detection while maintaining inspection records and providing dashboard analytics for efficient quality inspection monitoring.

8.1 Backend Integration Workflow

1. Receive the uploaded image from the React.js frontend.
2. Save the image in the **uploads** folder.
3. Perform image preprocessing.
4. Send the image to the trained AI model for prediction.
5. Generate the defect prediction and confidence score.
6. Store the inspection details in the MongoDB database.
7. Retrieve dashboard statistics from the database.
8. Return the prediction results to the frontend application.

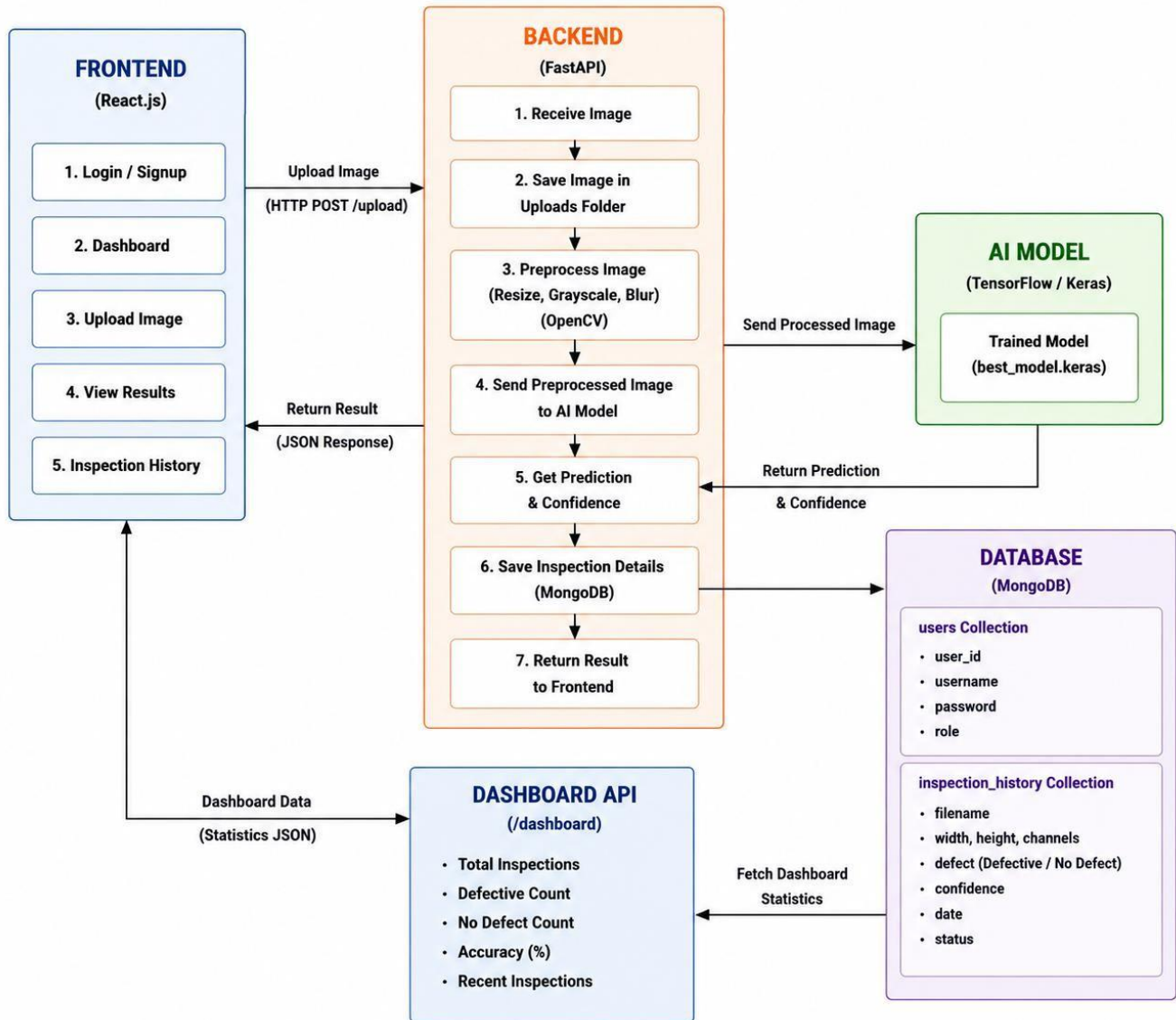


Figure 8.1: VisionInspect AI System Architecture

Figure 8.1 illustrates the overall architecture of the VisionInspect AI system. The React.js frontend communicates with the FastAPI backend to upload manufacturing images. The backend performs image preprocessing, sends the processed image to the trained AI model for prediction, stores inspection details in MongoDB, retrieves dashboard statistics, and finally returns the prediction results to the frontend.

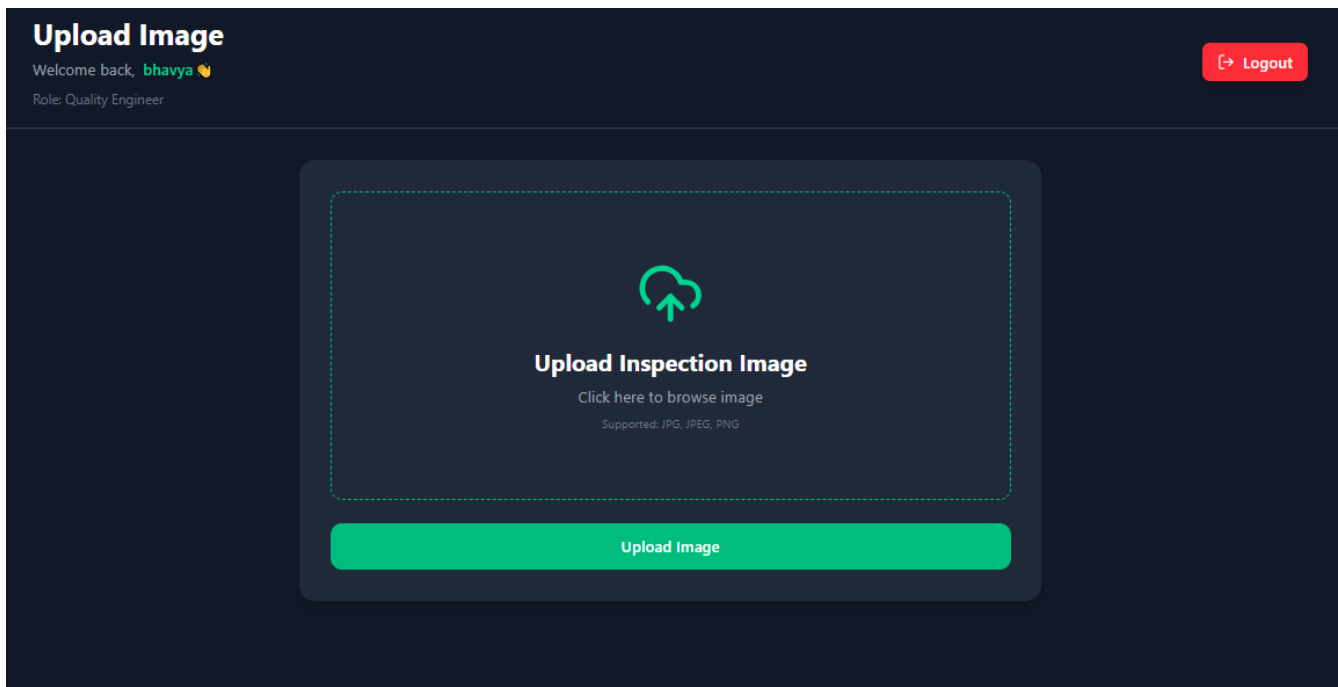
9. Frontend Implementation

The frontend of VisionInspect AI is developed using React.js to provide a user-friendly interface for manufacturing quality inspection. The application allows users to upload manufacturing images, view AI prediction results, download inspection reports, and monitor inspection statistics through an interactive dashboard. The frontend communicates with the FastAPI backend using REST APIs and displays the prediction results returned by the AI model.

9.1 Image Upload Module

The Image Upload module allows users to select manufacturing images from the local system for inspection. After selecting an image, the user uploads it through the application interface. The uploaded image is sent to the FastAPI backend, where image preprocessing and AI-based defect prediction are performed. Once the prediction is completed, the inspection results are returned and displayed to the user.

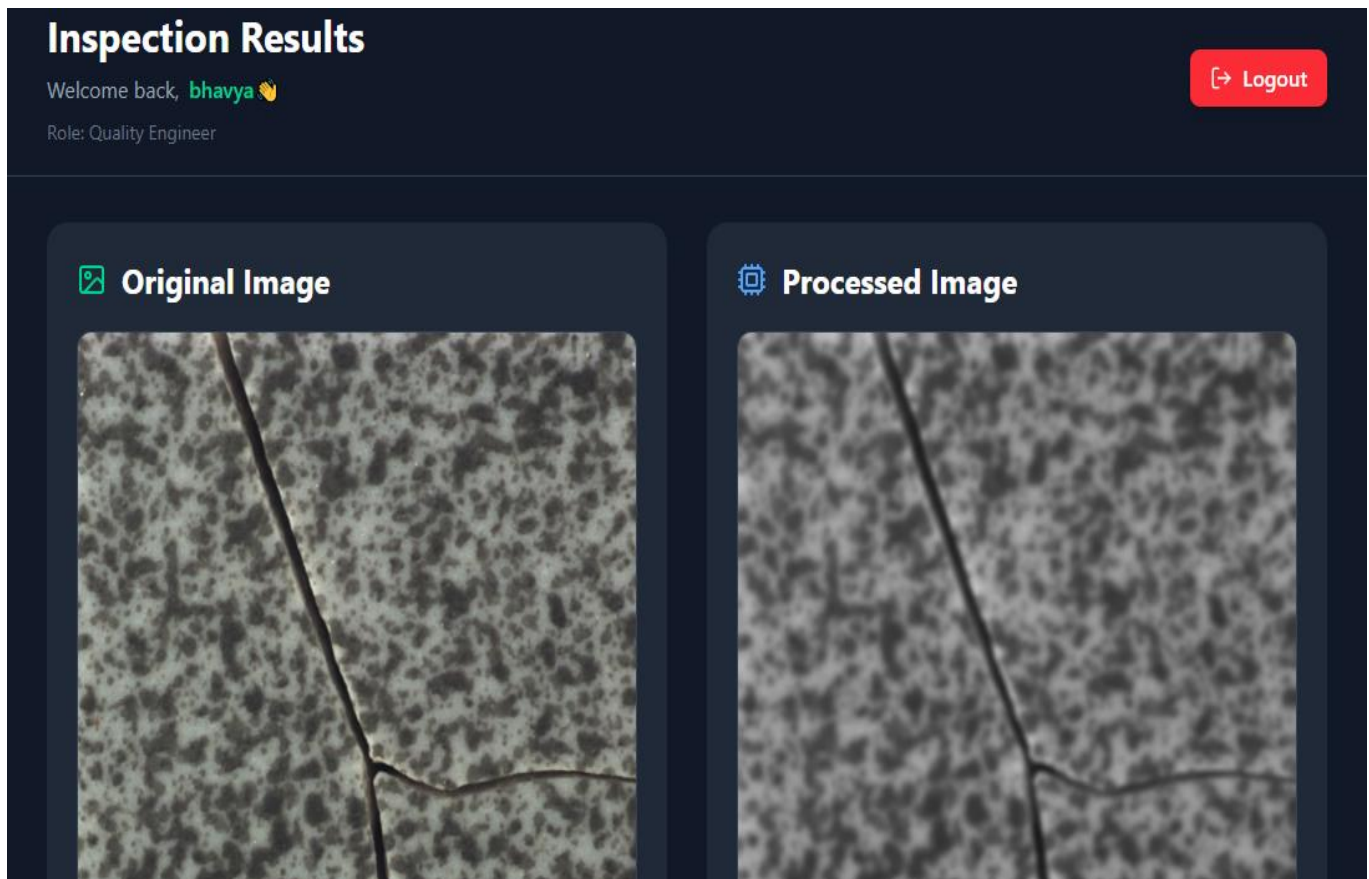
Output

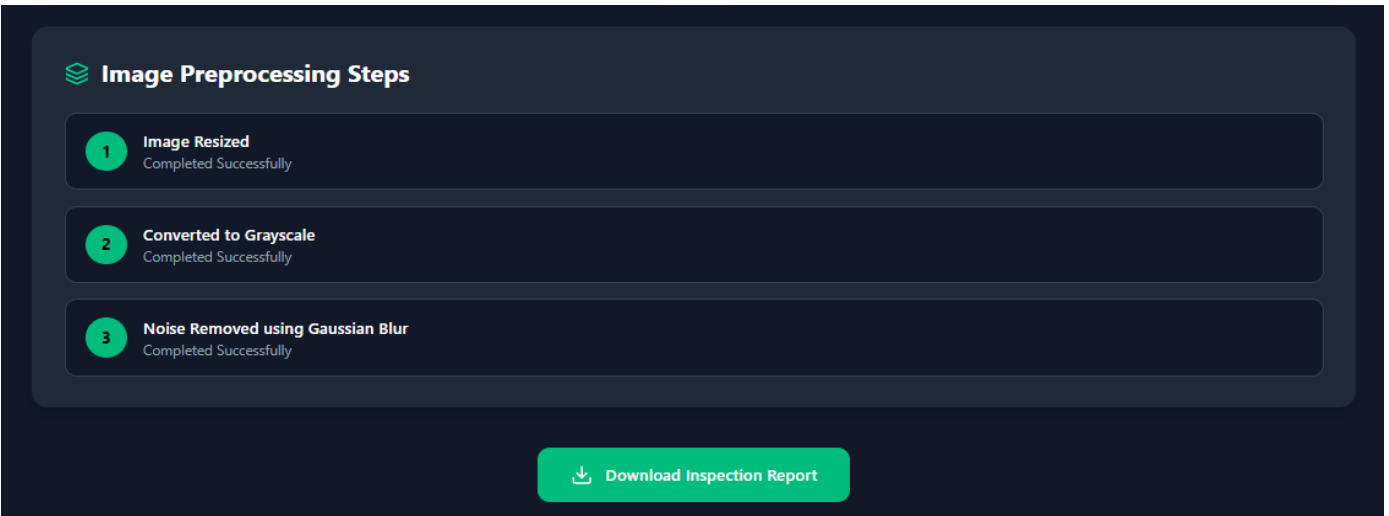
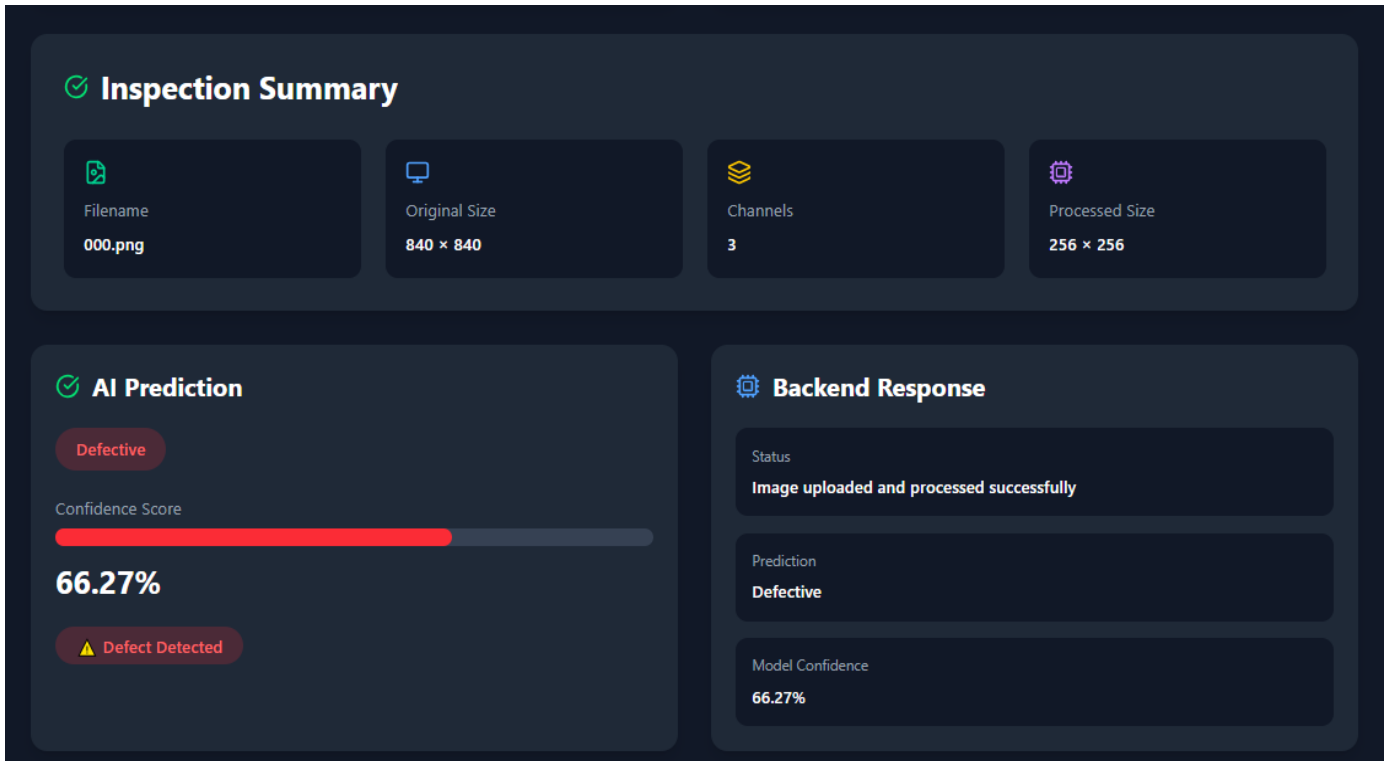


9.2 Inspection Results Module

The Inspection Results module displays the output generated by the AI model after analyzing the uploaded image. It presents both the original image and the processed image along with the predicted class, confidence score, preprocessing steps, and inspection summary. The module also provides an option to download the inspection report in PDF format for future reference.

Output

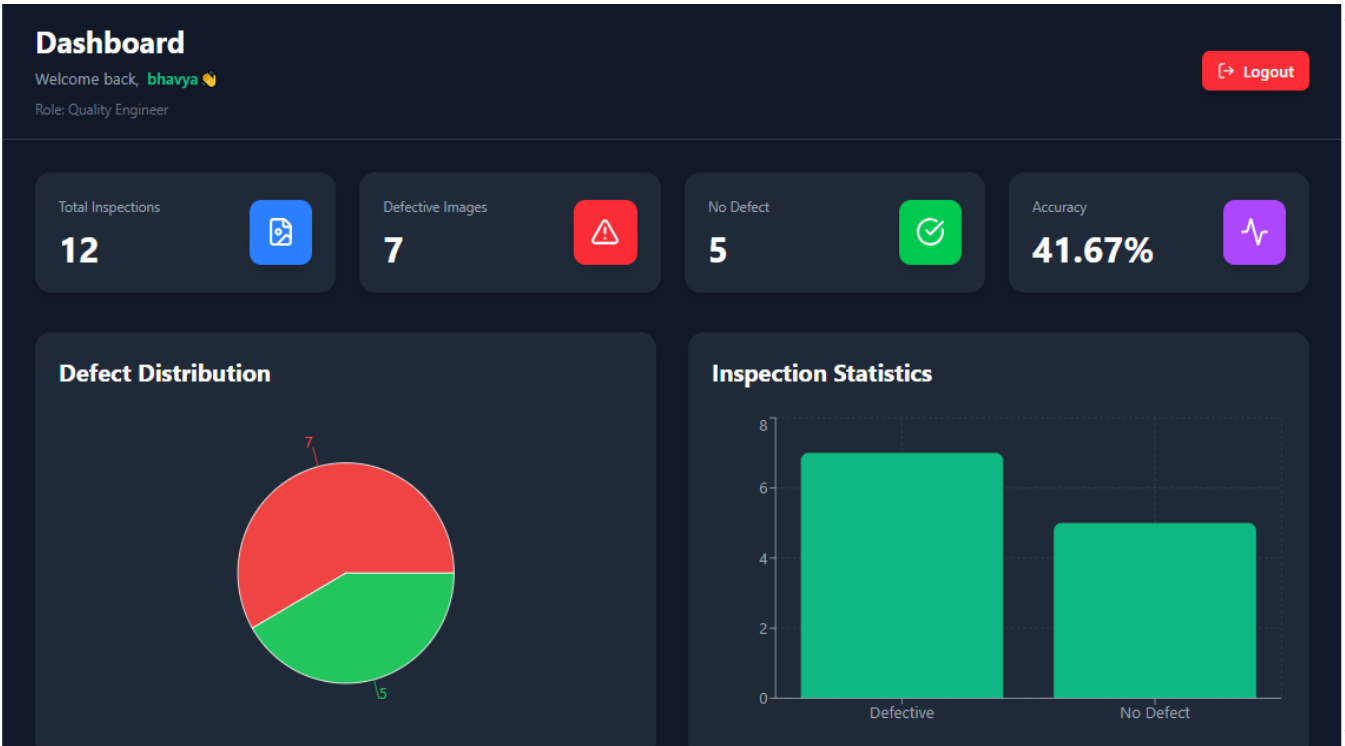




9.3 Dashboard Module

The Dashboard module provides a visual summary of the inspection process. It displays important statistics such as the total number of inspections, defective products, non-defective products, and overall model accuracy. It also includes graphical representations and recent inspection records to help users monitor the overall quality inspection process effectively.

Output



Recent Inspections

Filename	Defect	Confidence	Date
010.png	Defective	76.59%	24-07-2026 05:11 PM
014.png	Defective	64.19%	24-07-2026 05:10 PM
022.png	Defective	72.18%	24-07-2026 05:09 PM
001.png	Defective	70.29%	24-07-2026 04:21 PM
013.png	No Defect	86.73%	24-07-2026 04:19 PM

Dashboard Summary

Total Images Processed	Defective Images
12	7
No Defect Images	Model Accuracy
5	41.67%

9.4 Inspection History Module

The Inspection History module maintains a complete record of all previously inspected manufacturing images. It displays important details such as the image filename, prediction result, confidence score, and inspection date. The module provides a **search** feature to quickly locate inspection records using the filename and a **filter** option with **All**, **Defective**, and **No Defect** categories for easy classification of inspection results. Users can also **delete** unwanted inspection records and can also **export** the inspection history as a **CSV file** for reporting and further analysis. These features make it easier to manage, review, and analyze inspection data efficiently.

Output

Inspection History

Welcome back, **bhavya** 🍌

Role: Quality Engineer

Logout

Search by filename...

All

Export CSV

Filename	Status	Size	Defect	Confidence	Date	Action
000.png	Completed	1024 × 1024	Defective	73.28%	21-07-2026 07:00 PM	
008.png	Completed	1024 × 1024	No Defect	96.89%	21-07-2026 07:01 PM	
003.png	Completed	1000 × 1000	No Defect	83.84%	21-07-2026 07:27 PM	
013.png	Completed	1024 × 1024	Defective	59.61%	23-07-2026 09:36 PM	
016.png	Completed	840 × 840	Defective	69.39%	24-07-2026 12:42 PM	
021.png	Completed	1024 × 1024	No Defect	72.1%	24-07-2026 12:52 PM	

10. Conclusion

The **VisionInspect AI: Manufacturing Defect Detection & Quality Inspection System** was successfully developed by integrating **Artificial Intelligence, Computer Vision, FastAPI, React.js, and MongoDB**. The system is capable of preprocessing manufacturing images, classifying them as **Defective** or **No Defect** using a trained **MobileNetV2** deep learning model, and providing real-time prediction results through a simple and user-friendly web application.

The application also includes features such as image upload, dashboard analytics, inspection history management, search and filter options, CSV export, PDF report generation, and MongoDB integration. These features make the system more efficient and practical for manufacturing quality inspection.

Overall, the project demonstrates how Artificial Intelligence can automate the manufacturing quality inspection process by accurately classifying products, reducing manual effort, improving inspection efficiency, and supporting better quality control in manufacturing industries.

11. Future Enhancements

- Extend the current image classification model to support **object detection**, enabling the system to identify and locate defects in manufacturing images.
- Implement defect categorization and severity scoring for different types of manufacturing defects.
- Develop advanced manufacturing analytics dashboards with trend monitoring and production quality reports.
- Deploy the application for real-time industrial quality inspection.